

Module 14: Apache Kafka with Spring Boot

Questions

Q1. What is Apache Kafka, and why is it used?

Answer: Apache Kafka is a distributed event-streaming platform designed for high-throughput, fault-tolerant, real-time data pipelines and stream-processing applications. It combines messaging, storage, and stream processing in a single platform. Kafka is used because it can handle millions of events per second, store data durably for replay, decouple producers from consumers, and scale horizontally across clusters. Common use cases include event sourcing, log aggregation, real-time analytics, change data capture, metrics collection, and asynchronous communication between microservices.

Q2. What are the core components of Kafka?

Answer: The core components of Kafka are producers, consumers, topics, partitions, brokers, and ZooKeeper or KRaft controllers. Producers publish events to topics. Topics are logical streams of events. Topics are divided into partitions, which are ordered, append-only logs distributed across brokers. Consumers read events from partitions, often as part of a consumer group. Brokers are Kafka server nodes that store partitions and handle client requests. Controllers manage metadata such as topic creation, partition leadership, and replica placement. Modern Kafka uses KRaft mode, where controllers are Kafka nodes themselves, replacing the older ZooKeeper-based architecture.

Q3. What is a topic and a partition in Kafka?

Answer: A topic is a logical category or stream of events, similar to a table in a database but without enforced schema by default. A partition is a subdivision of a topic that acts as an ordered, immutable sequence of records. Each record in a partition gets a monotonically increasing offset that identifies its position. Partitions allow Kafka to scale horizontally because different partitions can be stored on different brokers and consumed by different consumers in parallel. Ordering is guaranteed only within a single partition, not across partitions of the same topic.

Q4. What is a Kafka broker, and how does it work?

Answer: A Kafka broker is a server node in a Kafka cluster. It receives and serves messages, manages storage, handles replication, and coordinates with other brokers through the controller. Each broker is identified by a unique broker ID. Producers connect to any broker to discover metadata about leaders and partitions, then send records directly to the broker that is the leader for the target partition. Consumers similarly fetch records from partition leaders. Multiple brokers form a cluster, and data is replicated across them for fault tolerance.

Q5. What is a producer in Kafka?

Answer: A producer is a client application that publishes records to Kafka topics. It sends records to the leader broker of the partition determined by the record key and partitioner strategy. Producers can configure acknowledgment levels, retries, batching, compression, and idempotence. By default, the producer uses a partitioner that hashes the key to choose a partition. If no key is provided, it uses a sticky or round-robin strategy depending on the Kafka version and configuration.

Q6. What is a consumer and a consumer group in Kafka?

Answer: A consumer is a client application that reads records from Kafka topics. Consumers subscribe to topics and fetch records from partitions. A consumer group is a set of consumers that cooperate to consume a topic. Kafka assigns each partition to exactly one consumer in the group, so records are load-balanced across group members. If there are more consumers than partitions, some consumers remain idle. If a consumer fails, its partitions are reassigned to other group members through a process called rebalancing.

Q7. What is an offset in Kafka?

Answer: An offset is a unique, monotonically increasing integer assigned to each record within a partition. It identifies the position of a record in the partition log. Consumers track their progress by committing offsets, which tells Kafka the last record they have successfully processed. Offsets can be committed automatically at fixed intervals, synchronously after processing, or manually by the application for precise control. The committed offset is stored in Kafka's internal `__consumer_offsets` topic.

Q8. What are the differences between Kafka and traditional message queues like RabbitMQ?

Answer: Kafka is a distributed log that stores events durably and allows multiple consumers to read the same data independently. Traditional message queues like RabbitMQ usually delete messages after delivery and focus on task distribution. Kafka scales horizontally through partitions, supports high throughput, and allows replay of events from the beginning. RabbitMQ is better for complex routing, priority queues, and request-response patterns. Kafka is better for event streaming, log aggregation, and decoupled event-driven microservices.

Q9. What are the main delivery semantics in Kafka?

Answer: Kafka supports three delivery semantics. At-most-once means a message is delivered zero or one time and may be lost. This happens when offsets are committed before processing. At-least-once means a message is delivered one or more times and may be duplicated if processing succeeds but offset commit fails. Exactly-once means each message is delivered and processed exactly one time. Exactly-once requires idempotent producers, transactions, and consumer-side handling such as storing output offsets together with processing results in a transactional store.

Q10. What is the role of replication in Kafka?

Answer: Replication makes Kafka fault-tolerant by copying partition data across multiple brokers. Each partition has one leader replica that handles all reads and writes, and one or more follower replicas that replicate the leader's log. The number of copies is controlled by the replication factor. If the leader fails, one of the in-sync replicas is elected as the new leader. Replication ensures that data is not lost when a broker crashes, but it adds some latency and storage overhead.

Q11. What is the ISR list in Kafka?

Answer: ISR stands for In-Sync Replicas. It is the set of replicas for a partition that are fully caught up with the leader. Only replicas in the ISR are eligible to become leaders if the current leader fails. A follower that falls too far behind is removed from the ISR. A producer configured with `acks=all` waits for acknowledgment from all ISR replicas before considering a record committed, which improves durability at the cost of latency.

Q12. What is the controller in Kafka, and how does KRaft change it?

Answer: The controller is the broker responsible for managing cluster metadata, including partition leadership, replica assignments, topic configuration, and broker registration. In the older ZooKeeper mode, the controller was elected by ZooKeeper and stored metadata in ZooKeeper znodes. In KRaft mode, Kafka uses its own Raft consensus protocol to manage metadata in an internal metadata log. A

small set of controller nodes form a quorum, and one active controller leads the metadata log. KRaft removes the ZooKeeper dependency, improves scalability, and enables faster controller failover because a new leader already has the metadata log replicated.

Q13. How does Kafka handle message ordering?

Answer: Kafka guarantees message ordering within a single partition. If a producer sends multiple records to the same partition, they are appended in the order they are received. If a consumer reads from that partition, it receives records in the same order. Ordering is not guaranteed across partitions. To preserve the order of related events, applications use the same key for those events so they all go to the same partition. For example, all events for a given order ID should use the order ID as the key.

Q14. What are Kafka partitions used for?

Answer: Partitions are the unit of parallelism in Kafka. They allow a topic to be split across multiple brokers, enabling horizontal scaling of both storage and throughput. Partitions also allow multiple consumers in the same group to process data concurrently, because each partition is assigned to only one consumer. However, more partitions increase the overhead of replication, leadership management, and consumer rebalancing, so the partition count should be chosen based on throughput needs and cluster capacity.

Q15. How do you install and run Kafka locally?

Answer: You download Kafka from the Apache Kafka website, extract the archive, and start ZooKeeper if using an older version, or start a KRaft broker in newer versions. For KRaft, you run `kafka-storage.sh random-uuid` to generate a cluster UUID, format the storage with `kafka-storage.sh format`, and start the broker with `kafka-server-start.sh`. You can create topics with `kafka-topics.sh`, send messages with `kafka-console-producer.sh`, and consume with `kafka-console-consumer.sh`. Confluent Platform and Docker images are also popular for local development because they bundle Kafka, Schema Registry, and control center.

Q16. What tools can be used to visualize and manage Kafka?

Answer: Common Kafka visualization and management tools include Confluent Control Center, Kafdrop, AKHQ, Kafka-UI by Provectus, Offset Explorer, and command-line scripts included with Kafka. Confluent Control Center provides monitoring, topic management, stream lineage, and alerting. Kafdrop and AKHQ

are lightweight web UIs for browsing topics, partitions, consumer groups, and offsets. These tools help developers inspect messages, understand lag, and diagnose issues without writing custom scripts.

Q17. How do you configure a Spring Boot application to produce messages to Kafka?

Answer: You add the `spring-kafka` starter dependency and configure properties such as `spring.kafka.bootstrap-servers`, `spring.kafka.producer.key-serializer`, and `spring.kafka.producer.value-serializer`. You can then inject `KafkaTemplate<K, V>` into a service and call `send(topic, key, value)` to publish records. Spring Boot auto-configures the producer factory based on properties. For JSON payloads, you set the value serializer to `JsonSerializer` or configure a custom serializer. You should also handle send results asynchronously using `ListenableFuture` or `CompletableFuture` and add error callbacks.

Q18. How do you consume messages from Kafka in Spring Boot?

Answer: You use the `@KafkaListener` annotation on a method and configure properties such as `spring.kafka.bootstrap-servers`, `spring.kafka.consumer.group-id`, `spring.kafka.consumer.key-deserializer`, and `spring.kafka.consumer.value-deserializer`. The annotated method receives the deserialized message, either as a String, a POJO, or a `ConsumerRecord`. Spring manages offset commits based on the `AckMode` setting. With `AckMode.RECORD`, the offset is committed after each record. With `AckMode.BATCH`, it is committed after a batch. For manual control, you can use `Acknowledgment.acknowledge()` and set `enable-auto-commit` to false.

Q19. What is `KafkaTemplate` in Spring Boot?

Answer: `KafkaTemplate` is a high-level Spring abstraction for sending messages to Kafka. It wraps a `ProducerFactory` and provides convenient methods such as `send`, `sendDefault`, and `executeInTransaction`. It returns a `ListenableFuture` or `CompletableFuture` that can be used to inspect the record metadata or handle failures asynchronously. `KafkaTemplate` simplifies producer configuration and integrates with Spring transaction management for exactly-once semantics when combined with `@Transactional`.

Q20. How do you configure multiple Kafka producers or consumers in Spring Boot?

Answer: You define multiple `ProducerFactory` or `ConsumerFactory` beans manually with distinct property prefixes and create separate `KafkaTemplate` or `ConcurrentKafkaListenerContainerFactory` beans. You then use `@Qualifier` or bean names to inject the correct factory where needed. For listeners, you specify the `containerFactory` attribute of `@KafkaListener`. This is useful when different topics need different serializers, security settings, or transaction configurations.

Q21. What is idempotence in Kafka producers, and why does it matter?

Answer: Idempotence means that sending the same record multiple times has the same effect as sending it once. Kafka producers can be configured with `enable.idempotence=true` to guarantee that duplicate retries do not produce duplicate messages within a single producer session. This is important because network failures can cause the producer to retry requests, and without idempotence, retries may result in duplicate records. Idempotence works by assigning sequence numbers to records per partition and tracking them on the broker.

Q22. What are the most important Kafka producer configuration properties?

Answer: Important producer properties include `bootstrap.servers`, `key.serializer`, `value.serializer`, `acks`, `retries`, `enable.idempotence`, `batch.size`, `linger.ms`, `compression.type`, `buffer.memory`, and `max.in.flight.requests.per.connection`. `acks` controls durability, with `all` being the safest. `batch.size` and `linger.ms` control throughput by grouping records. `compression.type` reduces network usage. `max.in.flight.requests.per.connection` affects throughput and ordering when retries occur.

Q23. What are the most important Kafka consumer configuration properties?

Answer: Important consumer properties include `bootstrap.servers`, `group.id`, `key.deserializer`, `value.deserializer`, `auto.offset.reset`, `enable.auto.commit`, `max.poll.records`, `max.poll.interval.ms`,

`session.timeout.ms` , `heartbeat.interval.ms` , and `fetch.min.bytes` .
`auto.offset.reset` determines behavior when no previous offset exists, such as reading from `earliest` or `latest` . `max.poll.records` limits the number of records returned per poll.
`max.poll.interval.ms` defines the maximum allowed time between polls before the consumer is considered failed and rebalanced.

Q24. What is the Kafka Schema Registry, and why is it used?

Answer: The Kafka Schema Registry is a service that stores and manages schemas for data used in Kafka topics. It is commonly used with the Confluent Platform and supports Avro, Protobuf, and JSON Schema. Producers register schemas with the registry and serialize messages using the schema ID. Consumers fetch the schema by ID to deserialize messages. The registry enforces compatibility rules such as backward, forward, and full compatibility, preventing incompatible schema changes from breaking producers or consumers. It enables centralized schema management, schema evolution, and data governance in event-driven systems.

Scenario based questions

S1. A Kafka producer sends messages but some records are lost during a broker restart. How do you prevent data loss?

Answer: I investigate the producer configuration and cluster setup. Data loss during a broker restart often happens because the producer uses `acks=1` , meaning it waits only for the leader acknowledgment and does not wait for replicas. If the leader crashes before followers replicate the data, the record is lost. I change the producer to `acks=all` so the producer waits for all in-sync replicas to confirm. I also verify that the topic has a replication factor of at least three and that `min.insync.replicas` is set to at least two. Additionally, I enable `enable.idempotence=true` to avoid duplicates during retries. I test the scenario by killing a broker during a load test and verifying no records are lost. Real-time example: a payment event producer writes to a topic with `replication.factor=1` and `acks=1` . When a broker restarts during deployment, some payment records vanish and downstream reconciliation fails. I recreate the topic with `replication.factor=3` , `min.insync.replicas=2` , update the producer to `acks=all` , and re-run the load test. The event loss drops to zero.

S2. A consumer group is rebalancing too frequently and causing delays. What do you check?

Answer: Frequent rebalances usually mean consumers are failing heartbeats, taking too long to process records, or crashing. I check `session.timeout.ms` and `heartbeat.interval.ms` to ensure they are appropriate for the processing time. I look at `max.poll.interval.ms`, which is the maximum time allowed between `poll` calls; if processing takes longer than this, the consumer is kicked out and the group rebalances. I reduce `max.poll.records` so each poll returns fewer records and processing finishes within the interval. I also check for exceptions causing consumer thread death and ensure consumer code does not block the poll loop. Real-time example: an order processing consumer downloads large files during message handling, causing `max.poll.interval.ms` to expire. The consumer is removed, partitions reassigned, and the same records reprocessed repeatedly. I move file downloads to a separate thread pool, reduce `max.poll.records` from 500 to 100, and set `max.poll.interval.ms` to 10 minutes. Rebalances stop and throughput stabilizes.

S3. A consumer is reading the same messages multiple times after a restart. What is happening?

Answer: This is likely at-least-once delivery with auto-commit enabled and offsets committed before processing completes. If the application restarts after receiving messages but before committing offsets, it will reprocess those messages on restart. I switch to manual acknowledgment by setting `enable.auto.commit=false` and the listener container `AckMode` to `MANUAL` or `MANUAL_IMMEDIATE`, then call `acknowledgment.acknowledge()` only after successful processing. For non-idempotent operations, I also deduplicate using a database unique key or idempotency cache. Real-time example: a notification service sends emails and uses auto-commit. After a deployment, it sends duplicate welcome emails to users. I switch to manual commit, store a `notification_sent` record in the database as part of the same transaction, and acknowledge the Kafka offset only after the insert succeeds. Duplicates stop even if the pod restarts mid-batch.

S4. Messages for the same customer are appearing out of order in a topic. How do you fix it?

Answer: Ordering is only guaranteed within a partition, so related messages must use the same key to land in the same partition. If the producer sends customer events without a key, they are distributed across partitions and may be consumed out of order. I ensure that all events for the same customer use the customer ID as the message key. I also check that there is only one consumer per partition in the group, because multiple consumers on the same partition is not possible in a consumer group. If stronger ordering is needed across many partitions, I reduce the partition count or use a custom partitioner. Real-time example: an e-commerce platform sends `OrderCreated`, `PaymentReceived`, and `OrderShipped` events without keys. A consumer sometimes sees

`OrderShipped` before `PaymentReceived` . I update the producer to use `orderId` as the key for all order-related events, guaranteeing that the entire order lifecycle stays in one partition and is processed in order.

S5. A topic is producing millions of messages per hour and a single consumer cannot keep up. How do you scale consumption?

Answer: I increase the number of partitions for the topic and add more consumers to the consumer group. Kafka assigns one partition per consumer, so the maximum parallelism equals the number of partitions. If the topic has only three partitions but the consumer group has one consumer, adding consumers will not help beyond three. After increasing partitions, I add consumers up to the partition count. I also optimize the consumer logic by reducing I/O, using batch processing, and avoiding synchronous calls. If the bottleneck is external, such as database writes, I consider buffering and writing in batches. Real-time example: a telemetry topic ingests sensor data and one consumer writes to a database. Lag grows continuously. I increase partitions from 3 to 24, add six consumer instances, and batch database inserts. Consumer lag drops from 500,000 records to near zero within minutes.

S6. A Spring Boot `@KafkaListener` is not receiving messages. How do you debug it?

Answer: I verify that the bootstrap servers are correct and reachable. I check the topic name spelling and that the consumer group ID is set. I look at the consumer logs for connection errors, authentication failures, or deserialization errors. If `auto.offset.reset` is `latest` and no new messages are produced, the listener will appear idle. I use `kafka-console-consumer.sh` or a UI tool to confirm messages exist. I check the container factory configuration, including key and value deserializers. If a custom error handler is configured, I verify it is not silently swallowing exceptions. Real-time example: a new microservice's listener uses `spring.kafka.consumer.group-id=order-processor-local` while production expects `order-processor` . Messages are consumed by the wrong group and the new service sees none. I align the group ID, verify offsets in the UI, and add a startup health check that fails if the listener container is not assigned partitions.

S7. A Kafka cluster is running out of disk space. What are the likely causes and fixes?

Answer: Causes include unbounded topic retention, large message payloads, high replication factor, or log compaction not being enabled where appropriate. I check retention settings such as `retention.ms` , `retention.bytes` , and `segment.bytes` . I identify high-volume topics and

either reduce retention, enable compaction, or add more storage. I also verify that consumers are keeping up, because lagging consumers cause Kafka to retain messages longer than intended. For long-term storage needs, I move older data to object storage using Kafka Connect or tiered storage if available. Real-time example: a logging topic retains data for 30 days and accumulates 10 TB. Analytics only needs 7 days. I change `retention.ms` to 7 days for the analytics topic and enable tiered storage for audit logs. Disk usage drops by 70 percent and the cluster avoids an outage.

S8. You need to guarantee exactly-once processing for a Kafka consumer that writes to a database. How do you design it?

Answer: I use Kafka transactions for exactly-once semantics. The producer is configured with `transactional.id` and `enable.idempotence=true`. The consumer has `isolation.level=read_committed`. I wrap the database write and the offset commit in a single transaction using `@Transactional` or `kafkaTemplate.executeInTransaction`. If the database transaction fails, the Kafka offset is not committed, so the records can be reprocessed. For the consumer side, I prefer storing the processed offset in the same database table as the business data using a unique identifier derived from the topic, partition, and offset. This makes processing idempotent without needing Kafka transactions on the consumer. Real-time example: a fraud detection service writes alerts to PostgreSQL. To avoid missing or duplicating alerts, I store the alert along with `kafka_topic`, `partition`, and `offset` in a unique composite key. On restart, duplicate records are rejected by the database constraint, achieving exactly-once behavior.

S9. A producer is experiencing high latency. What Kafka settings can improve throughput?

Answer: I tune producer settings for batching and compression. I increase `batch.size` so more records are sent together, and increase `linger.ms` so the producer waits briefly to fill batches. I enable compression with `compression.type=lz4` or `snappy` to reduce network traffic. I increase `buffer.memory` if the producer is producing faster than the broker can acknowledge. I also verify that `acks` is not set to `all` if lower durability is acceptable, though this is a trade-off. If the bottleneck is broker-side, I add more partitions or brokers. Real-time example: a real-time bidding producer sends one record at a time and has 200ms p99 latency. I set `linger.ms=20`, `batch.size=65536`, `compression.type=snappy`, and `buffer.memory=67108864`. Latency drops to 30ms and throughput increases tenfold.

S10. A Kafka topic must support both a fast consumer and a slow batch consumer. How do you handle this?

Answer: I use separate consumer groups for each consumer. Kafka allows multiple consumer groups to read the same topic independently, each maintaining its own offsets. The fast consumer uses a small `max.poll.records` and commits frequently. The slow batch consumer uses a larger `max.poll.records`, longer `max.poll.interval.ms`, and commits after each batch. This decouples their processing speeds. If needed, I can also use Kafka Streams or Connect to fan out data to specialized topics for each consumer type. Real-time example: a recommendation service needs near-real-time events, while an analytics service builds daily reports. I create two consumer groups: `recommendation-fast` with `max.poll.records=50` and `analytics-batch` with `max.poll.records=1000`. Each progresses at its own pace without affecting the other.

S11. A consumer processes messages and calls an external HTTP service. The external service is slow and the consumer is kicked out of the group. What do you do?

Answer: I move the external HTTP call out of the polling thread. I use a thread pool to process records asynchronously and pause polling if the queue fills up. Alternatively, I reduce `max.poll.records` and increase `max.poll.interval.ms` to give more processing time. The cleanest solution is to offload the external call to a separate worker stage, either by publishing to a worker topic or by using a buffering queue with pause and resume logic. I also add timeouts and circuit breakers around the external call. Real-time example: a shipping consumer calls a carrier API that occasionally takes 30 seconds. Consumers are evicted and records reprocess. I implement an in-memory queue, process the carrier calls in a separate thread pool, and pause the listener when the queue exceeds 100 items. The consumer stays in the group and reprocessing drops.

S12. You need to replay messages from a Kafka topic from the beginning for a new service. How do you do it?

Answer: I deploy the new service with a unique consumer group ID and set `auto.offset.reset=earliest`. Because the group has no committed offsets, it starts from the beginning of each partition. If the topic has retention limits, some older messages may already be gone, so I verify retention settings first. For a one-time backfill, I can also use `kafka-consumer-groups.sh --reset-offsets` to move an existing group's offsets to earliest. I ensure the new consumer is idempotent so replay does not cause duplicate side effects. Real-time example: a new reporting service needs a full year of order events. I create a new consumer group `reporting-`

`backfill` , set `auto.offset.reset=earliest` , and scale up consumers to match partitions. The service replays a billion records over a weekend and populates its data warehouse.

S13. A Kafka topic currently has six partitions, but business growth requires twelve. How do you add partitions?

Answer: I use `kafka-topics.sh --alter --topic <topic> --partitions 12` to increase the partition count. Kafka redistributes existing data only for new messages; existing segments stay on their current brokers unless I run a partition reassignment. After adding partitions, I review consumer group assignment to ensure all consumers are used. I also consider the impact on keyed messages: new keys may map to new partitions, and ordering is still guaranteed per partition. For topics with strict ordering, I plan partition expansion carefully and possibly repartition data using a temporary topic. Real-time example: a checkout topic grows from 6 to 12 partitions to handle Black Friday traffic. I run the alter command, update the consumer deployment to 12 pods, and verify balanced partition assignment. I also update dashboards and alerting because key-based ordering now spans more partitions.

S14. A producer and consumer use JSON serialization, but the consumer fails with deserialization errors after a producer change. How do you prevent this?

Answer: I introduce a Schema Registry with Avro, Protobuf, or JSON Schema to enforce compatible schema evolution. Producers register schemas and consumers fetch compatible schemas by ID. I set a compatibility mode such as `BACKWARD` or `FULL` so breaking changes like renaming fields or changing types are rejected. If I must keep JSON, I use a stable schema with additive changes only and configure the consumer to ignore unknown properties. I also version events or use separate topics for incompatible formats. Real-time example: a producer changes a date field from String to Long timestamp. The JSON consumer crashes with `NumberFormatException` . I migrate the topic to Avro and Confluent Schema Registry, define the schema with a union type, and enforce backward compatibility. Future changes are validated in CI before deployment.

S15. A Schema Registry enforces backward compatibility, but a developer removes a required field and the schema is rejected. What should the developer do?

Answer: The developer should not remove required fields when backward compatibility is required, because existing consumers expect that field. If the field is truly no longer needed, the developer can mark it as optional with a default value, or add a new schema version that deprecates the field. In some cases, a topic migration or a new major version topic is the correct path. The safest approach is to add fields rather than remove them, and to test schema changes against the registry's compatibility endpoint in CI. Real-time example: a developer removes the `customerEmail` field from an Avro order schema. The Schema Registry rejects the new version because active consumers still read `customerEmail`. I help the developer add the field back as optional with a default null value, update the consumers to stop using it, and then, once all consumers are updated, remove the field in a later version.

S16. A Kafka cluster using ZooKeeper needs to be migrated to KRaft. What do you consider?

Answer: I verify the Kafka version supports migration, such as Apache Kafka 3.4 or later. I plan the migration using the online migration process described in KIP-866, which includes dual-write mode for rollback. I back up ZooKeeper metadata, provision a KRaft controller quorum, and enable metadata migration on both ZooKeeper and KRaft nodes. I migrate brokers in a rolling fashion, monitor metadata consistency, and finalize the migration once all nodes are on KRaft. Because ZooKeeper mode is removed in Kafka 4.0, migration is mandatory for upgrades beyond that version. I also update operational runbooks and monitoring to remove ZooKeeper dependencies. Real-time example: a company runs Kafka 3.5 with ZooKeeper and wants to upgrade to Kafka 4.0. I set up three KRaft controllers, enable dual-write, roll brokers to KRaft mode over several maintenance windows, and finalize once metrics and partition assignments are stable. ZooKeeper is then decommissioned.

S17. A Kafka producer receives timeout errors under heavy load. How do you investigate?

Answer: I check broker metrics for CPU, disk, and network saturation. I review producer metrics for `record-queue-time-avg`, `request-latency-avg`, and `buffer-available-bytes`. If the buffer is full, the producer blocks waiting for network threads to catch up. I increase `buffer.memory` or reduce `linger.ms` if batches are forming too slowly. I also check whether the topic has enough partitions and whether some brokers are hot spots due to uneven partition distribution. If all brokers are healthy, the issue may be a slow consumer causing retention pressure, but that usually affects brokers rather than producer timeouts. Real-time example: during a flash sale, producers start throwing timeout exceptions. Broker CPU is at 95 percent because partition leadership is concentrated on two brokers. I run a partition reassignment to spread leaders evenly, add two brokers, and tune `buffer.memory`. Producer timeouts disappear and throughput doubles.

S18. A consumer group is stuck and not making progress. How do you find the root cause?

Answer: I check consumer lag using `kafka-consumer-groups.sh --describe` or a UI. If lag is growing, the consumer is slower than the producer. I check application logs for exceptions, look at processing time per record, and inspect `max.poll.interval.ms` for evictions. If no partitions are assigned, the group may have more consumers than partitions, or a rebalance is in progress. I also verify that the consumer is not stuck on a poison pill record by checking error handlers and dead-letter topic configuration. If the topic has no new data, the consumer is correctly idle. Real-time example: a payment reconciliation consumer shows increasing lag. The logs show no errors, but each record triggers a 2-second database query. I batch the database lookups, reduce query time to 50 milliseconds per batch, and lag clears within an hour.

S19. You want to use Kafka for event sourcing. What design choices do you make?

Answer: I use topics as event streams, with each aggregate type having its own topic. Events are immutable and appended to partitions keyed by aggregate ID to preserve ordering. I version events, include metadata such as event type and timestamp, and avoid deleting or updating events. Consumers build read models by replaying events. I use log compaction for state-snapshot topics so the latest state per key is retained. I also design consumers to be idempotent and handle replays safely. Schema Registry is important to maintain event schema compatibility over time. Real-time example: an inventory system uses event sourcing where each product has its own event stream keyed by `productId`. Events include `StockReceived`, `StockReserved`, and `StockShipped`. A separate projection service replays the topic to build a read-optimized view in Elasticsearch. If the projection needs rebuilding, I reset the consumer group to earliest and replay all events.

S20. A Spring Boot application must publish to and consume from Kafka with Avro and Schema Registry. How do you configure it?

Answer: I add dependencies for `spring-kafka`, `kafka-avro-serializer`, `kafka-avro-deserializer`, and the Avro Maven or Gradle plugin to generate Java classes from schemas. I configure `spring.kafka.producer.value-serializer` to `io.confluent.kafka.serializers.KafkaAvroSerializer` and `spring.kafka.consumer.value-deserializer` to `KafkaAvroDeserializer`. I set `schema.registry.url` for both producer and consumer. For consumers, I set

`specific.avro.reader=true` to use generated classes instead of generic records. I generate Avro classes from `.avsc` files during the build. I also configure compatibility mode in the Schema Registry, typically `BACKWARD`, and set `auto.register.schemas=false` in production to prevent accidental schema registration. Real-time example: an order service publishes `OrderCreated` Avro events to a topic. The build generates `OrderCreated.java` from `OrderCreated.avsc`. The producer serializes using `KafkaAvroSerializer`, which registers the schema if needed and embeds the schema ID. A downstream fulfillment service uses `KafkaAvroDeserializer` with `specific.avro.reader=true` and fetches the schema by ID. CI validates schema changes against the registry before deployment, ensuring compatibility.

S21. A Kafka topic receives events from two producers with different Avro schemas. How do you handle this?

Answer: I use Schema Registry subjects and schema references to support multiple event types in the same topic. One approach is to define a wrapper schema such as an envelope with a type discriminator, or to use Schema Registry's support for multiple schemas per subject if the registry and clients support it. Another approach is to separate event types into different topics if they have different schemas and lifecycles. If I keep a single topic, I ensure the consumer can handle all expected schemas, often by using generic Avro records and routing based on a type field. I also enforce compatibility carefully because mixing schemas in one topic complicates evolution. Real-time example: a `user-events` topic receives both `UserRegistered` and `UserProfileUpdated` events. I create an Avro union type or an envelope schema with a `type` field and a union payload. The Schema Registry stores both referenced schemas, and the consumer uses the type field to cast the payload to the correct generated class.